

Chromium on navi

How a browser engine became part of the operating system - and why navi intends to own more of that layer.

On navi, the browser is not just an app. It is an application runtime, a policy surface, a visual layer, and eventually a platform we intend to shape ourselves.

The web is the ideal app layer for navi: portable, inspectable, offline-capable when we build it that way, and already understood by the tools people use every day.



The browser stack is already real; ownership is the next step

72

shipped webapp launchers

Every launcher uses Chromium app mode and forces dark rendering.

4

control surfaces

Runtime, launchers, policy, and optional browser installers.

1

long-term browser

accela: a planned xvoidsx Chromium fork, not a near-term release promise.

What exists today

- Debian Chromium as navi's default browser.
- `navi-webapp` for install, uninstall, list, and doctor workflows.
- Rofi-visible .desktop launchers that open sites and local pages as app windows.
- A wrapper that makes dark mode deterministic on sway.
- Managed policy for nightshadeNeon, uBlock Origin Lite, and Proton Pass.
- Optional browser installers in `navi-extras`, including Helium and Google Chrome.

What is being tested or planned

- **Helium:** daily-driver dogfood and a leading candidate for accela's base.
- **Microsoft Edge:** an optional browser whose Linux bugs are being pushed upstream.
- **blackice:** dogfooding now; it has not replaced uBlock Origin Lite in policy.
- **Browser picker:** a planned settings TUI for choosing the Chromium-family webapp runtime.
- **accela:** the eventual owned browser layer, after nearer-term navi work.

The point is not Chromium for its own sake. The point is a stable web runtime that navi can integrate deeply without taking choice away.

Contents

03 The webapp runtime

04 Local-first behavior

05 Managed policy

06 Browser lineup

07 Edge on navi

08 Upstream citizenship

09 blackice

10 Why Helium leads

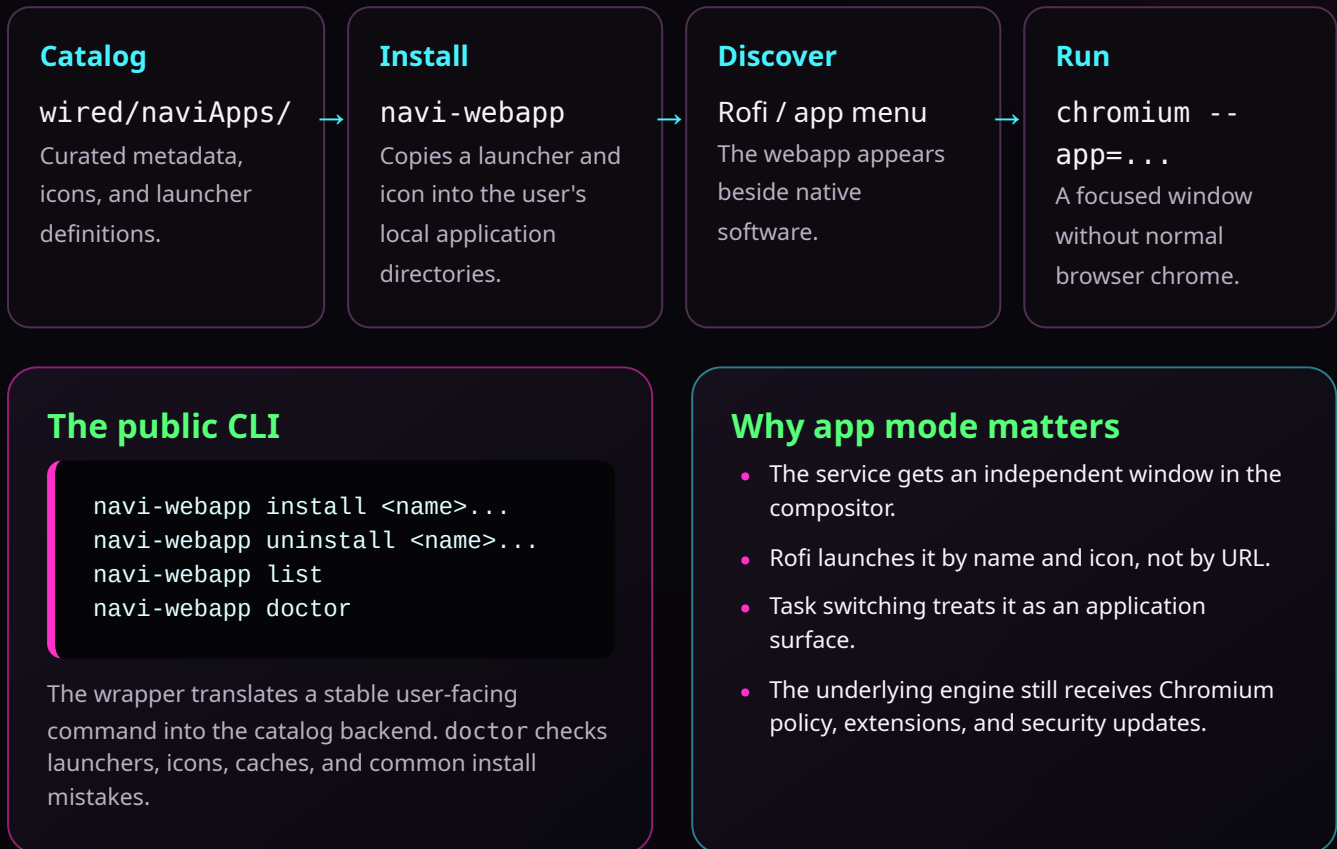
11 accela's product layer

12 Roadmap and guardrails

13 Appendix and sources

navi turns the web into first-class desktop software

A navi webapp is not a bookmark. It is an installed desktop entry with its own icon, its own window, and the same launcher visibility as a native application.



First-party front doors

navi radio, neighborli, glyph, and the naviApps storefront make the strategy visible: the distro's own services can arrive as web technology without feeling bolted on. Users can then add more from the 72-launcher catalog. The default install is curated rather than exhaustive; the wider catalog remains available on demand.

The current default set also includes selected community and commercial services. Telegram and Element have moved toward native-app paths, showing that navi treats the web layer as a strong default mechanism rather than a dogma.

Local-first engineering keeps the app layer dependable

navi's web stack includes pages that never need to become public internet services. A local page should work as a local page, including when Chromium is already running.

The failure that shaped the design

The naviApps storefront originally read its catalog with `fetch()`. Under `file://`, that required a file-access flag. Chromium can ignore new command-line flags when an existing browser process handles the request, so the store could fail in the most ordinary case: the browser was already open.

The fix was architectural, not another launch flag. `apps.json` remains the source of truth, but `build-catalog.sh` emits an inlined `catalog.js`. The local page loads that data directly and no longer depends on file-origin fetch behavior.^[2]

Design rules that fell out of it

- **Offline-first means no hidden server.** Local tools must work from the filesystem.
- **Existing browser state counts.** A clean launch test is not enough.
- **Flags are part of the runtime contract.** All 72 launcher files currently carry both `--app` and `--force-dark-mode`.
- **Generated data must ship with its source.** Catalog changes update both `apps.json` and `catalog.js`.

Deterministic dark mode

Chromium's system-theme detection can be unreliable on sway when the desktop portal is not answering. navi therefore puts a wrapper at `/usr/local/bin/chromium` that calls Debian's browser with `--force-dark-mode`. A generated desktop override routes menu launches through that wrapper without modifying Debian's packaged file.



User control remains: because navi's flag comes first, an explicit later `--force-light-mode` can override it. A curated default is not the same as a lock.

Policy makes the baseline coherent across profiles

navi uses Chromium's managed policy layer to install a small, explicit set of browser components. The result is reproducible: a new profile still looks and behaves like navi.

DEPLOYED

nightshadeNeon

The browser theme arrives on first launch and stays present while policy applies.

DEPLOYED

uBlock Origin Lite

The current policy-level blocker. blackice has not replaced it yet.

DEPLOYED

Proton Pass

The stable extension is pinned as part of the baseline.

The policy is stored as JSON under `/etc/chromium/policies/managed/` and uses Chromium's `ExtensionInstallForcelist`. Force-installed items are installed silently and cannot be disabled or removed by the user while the policy remains in effect.^[3]

What this buys navi

- **Coherence:** theme and security defaults do not depend on profile sync.
- **Privacy by default:** blocking is present before a user has to shop for an extension.
- **Supportability:** bug reports begin from a known baseline.
- **Recovery:** the baseline is reconstructed by redeployment, not by profile surgery.

What this obligates navi to do

- **Keep the list small.** A non-removable default carries more responsibility than a recommendation.
- **Pin the right channel.** Stable IDs only; no beta surprise.
- **State ownership honestly.** uBOL is upstream; blackice is a fork; Proton Pass is Proton's.
- **Test browser coverage.** Policy directories differ across Chromium-family packages.

Coherence is useful only when the user can understand what is being enforced and why.

One runtime contract, several browser choices

navi can be opinionated about the integration layer without pretending there is one correct browser for everyone.

DEFAULT TODAY

Chromium

Debian's Chromium is the known baseline. navi's wrapper, launcher overrides, app windows, and policy are designed around it and exercised by the default install.

DOGFOODING

Helium

Helium is an open-source Chromium browser maintained by imput, based on ungoogled-chromium and shipped with its own Linux packaging.^[4] Raven is using it as a daily driver while navi evaluates it as the likely base for accelar. The questions are practical: security-update lag across releases, package behavior on navi, and whether the divergence list stays short.

CANDIDATE OPTION

Microsoft Edge

Edge is being evaluated as a user choice, not a default. Its Linux build looks at home under nightshadeNeon and its performance controls are unusually granular. Its telemetry model does not match navi's zero-telemetry direction, so that tradeoff belongs in the recommendation rather than in fine print.

OPTIONAL PATH

Other browsers through navi-extras

The post-install menu already exposes user-driven browser additions such as Helium, Google Chrome, and Brave Nightly. Installer scripts live under `scripts/installers/`, deploy to `/usr/share/navi/installers/`, and are never run by the main installer. One optional browser failure must not endanger the operating-system install.

Planned: a browser picker in navi settings

The picker would detect installed Chromium-family browsers, show the current webapp runtime, test app-mode support, and change one canonical setting. Webapps would resolve that setting instead of hardcoding chromium. The scope stays Chromium-based because app mode and managed-policy behavior are the contract navi can support consistently.

Edge has a real efficiency case - and visible Linux debt

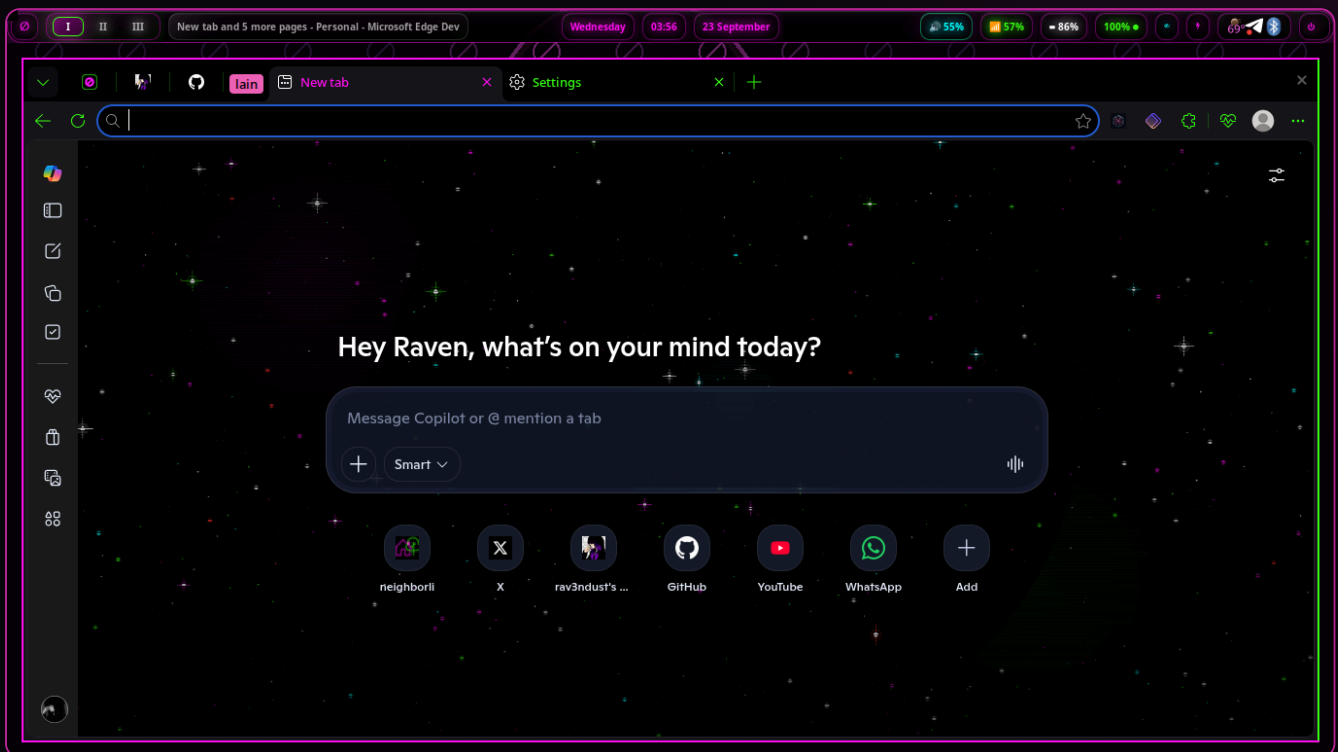


Figure 1. Microsoft Edge Dev on navi with nightshadeNeon applied. The fit is not hypothetical: Edge already participates in the distro's visual language.

Why it is worth supporting

Edge exposes sleeping-tab controls that put inactive tabs to sleep, lets users choose the inactivity interval, and allows exceptions. Microsoft describes the feature as a way to reduce memory and CPU use; its performance settings also include efficiency controls.^[5]

That is not a benchmark claim for navi hardware. It is a product capability with obvious relevance to low-power systems and machines with limited memory.

Why it is not the default

navi's direction is zero telemetry. Edge's product and account stack makes it a tradeoff rather than a clean philosophical fit. The honest position is stronger: users who value Edge's workflow or efficiency controls should be able to choose it, while navi states clearly why it remains optional.

Integration bar: visual fit is necessary but not sufficient. A browser option also needs dependable Linux window behavior, working first-party controls, predictable updates, policy compatibility, and app-mode launches that survive real daily use.

Integration includes trying to make upstream Linux support better

navi's first response to a browser bug should not always be a distro-only patch. When the failure belongs upstream, the useful work is evidence, a clear report, and verification when a fix lands.

TRACKER ID	OBSERVED BEHAVIOR	STATE
EDGE-LINUX-001	The Copilot toolbar button does not function on the tested Linux build.	Open; awaiting response
EDGE-LINUX-002	Enabling "Use system title bar and borders" produces no change; Edge's custom chrome remains in place.	Open; awaiting response
23 Sep 2026	Evidence sent upstream. Raven submitted both issues through Edge's built-in feedback system with a screenshot, a screen recording, and a detailed write-up. He identified himself as a navi maintainer who wants Edge to be a reliable distro option.	
Now	Public record maintained. docs/edge-linux-feedback.md keeps the issue IDs, exact behavior, submission history, and current status in one place. ^[6]	
When fixed	Verify on navi. The tracker calls for testing on eiri's Debian trixie base, against Edge Dev and Stable, on real navi hardware. A vendor response is not the same as a verified fix.	

A distro maintainer lobbying Microsoft for better Linux support is unusual leverage. It should be spent deliberately: reproducible behavior, visual evidence, one public status page, and no inflated claims.

Fix upstream when possible

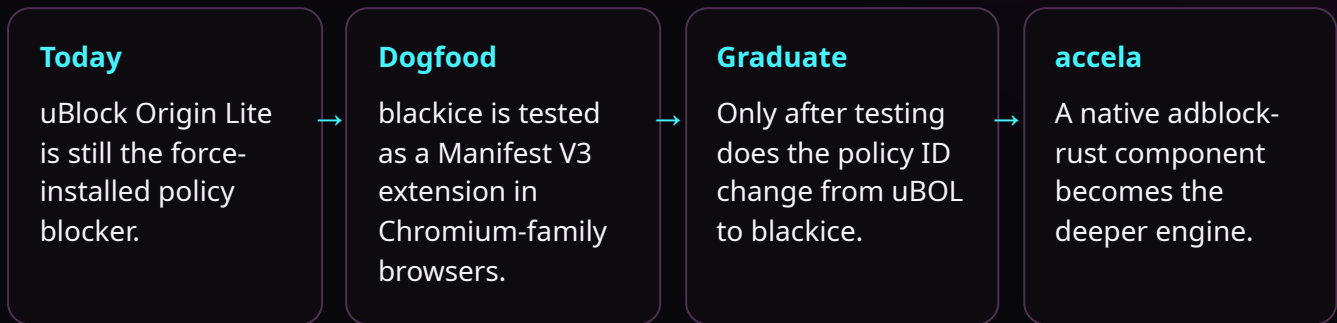
A private patch shifts maintenance onto navi and hides the Linux problem from everyone else. An upstream repair improves Edge for other distributions too.

Dogfood is due diligence

The same principle applies to Helium. Daily use is how update lag, policy gaps, regressions, and browser-specific assumptions become visible before accelera inherits them.

blackice moves blocking from a chosen extension toward owned infrastructure

The transition is staged on purpose. navi should replace a proven blocker only when its own fork has earned the job.



What blackice is

blackice is xvoidsx's GPLv3 fork of uBlock Origin Lite: a Manifest V3 ad and tracker blocker with a nightshadeNeon interface. The blocking engine, filter-list machinery, and Declarative Net Request conversion remain upstream work.^[7]

Raymond Hill's copyright and the uBlock Origin contributors' notices stay intact; xvoidsx adds its copyright only to its own interface, branding, defaults, and genuine curated-list work.

Boundaries that protect credibility

- Never present blackice as an official uBlock project.
- Never dogfood it beside uBO or uBOL; overlapping blockers fight over the same requests.
- Never show fake statistics, controls, or placeholder lists.
- Keep the Chromium build as the supported target until the Firefox path is packaged and tested.
- Change the navi policy only after the dogfood gate passes.

One brand, two engines

The extension is the portable phase: it can run in Chromium-family browsers now and build recognition around one privacy surface. The native accela phase is different work. An adblock-rust component can sit closer to the browser and avoid making the extension the permanent architectural ceiling. The name and user experience can remain continuous even as the engine changes.

Helium is the practical starting point, not a dependency of faith

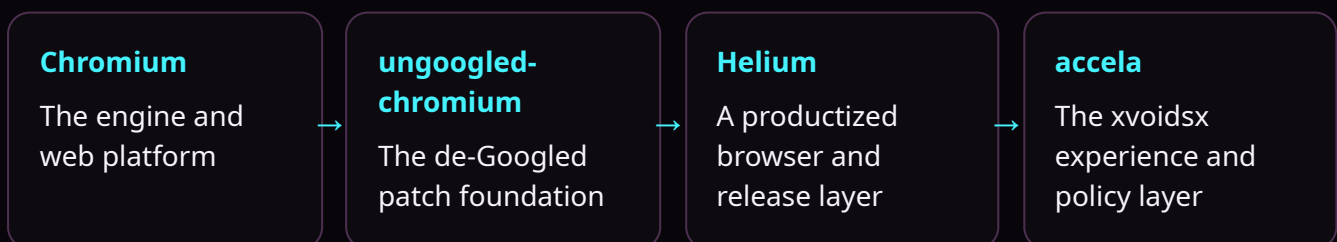
Forking raw ungoogled-chromium would maximize control on day one and maximize release-engineering burden at exactly the same time. Helium changes that equation.

Why Helium leads

- **Productized base:** Linux packaging, update paths, services, and cross-platform work already exist in public repositories.
- **Ungoogled foundation:** Helium is based on ungoogled-chromium rather than stock Google Chrome.
- **Patch discipline:** accelacan aim to remain a legible patch set instead of becoming an opaque Chromium tree.
- **Open source:** Helium-specific work is published under GPL-3.0, while imported components retain their licenses.^[4]

What navi must watch

- **Security-update lag:** dogfood across several Chromium releases, not one good week.
- **Divergence:** record every “Helium chose X; accelawould choose Y” decision.
- **Ad blocking:** Helium's baked-in uBlock component and blackice are the clearest known overlap.
- **Linux integration:** policy directories, app mode, theming, codecs, and window behavior need target-system tests.



The escape hatch is part of the design

If Helium's direction or cadence stops fitting, a clean accelacpatch stack should be portable back toward plain ungoogled-chromium. A base is useful when it reduces work without becoming a trap.

Helium's own README describes it as beta. That makes Raven's daily-driver period a precondition, not a ceremonial trial: normal browsing, updates, extensions, app windows, and navi policy all need time to fail honestly.

The goal is a navi-native browser, not a logo swap

accela is where the browser becomes an owned interface to the wired: visual identity, privacy, search, agents, and the smallweb designed as one system.

Smallweb built in

Gemini and other smallweb paths become first-class navigation, not specialist afterthoughts.

Zero telemetry

No analytics or product surveillance added by xvoidsx; privacy is an architecture constraint.

Native blackice

adblock-rust moves blocking into the browser while the extension remains the portable phase.

Hey Lain

Local and cloud agent support replaces the generic "AI Mode" story with a navi-native assistant surface.

Private search

A privacy-respecting search layer with a smallweb-scoped index as the long-term wedge.

Native bangs

!radio, !n1, !apps, and !wired turn the address bar into a map of navi's own front doors.

"Wallpaper is the theme"

Chromium's native color extraction can use a new-tab-page wallpaper to tint the wider browser chrome. In Raven's navi tests, pulsegrid, katakana rain, and jellyfish each produced coherent palettes without the muddy mappings seen with an external theme. That suggests a stronger accela model: the selected navi gifpaper becomes the browser's visual source of truth.

Leading default: pulsegrid

The boldest statement of the navi visual language. Use a static frame by default on older hardware; animation can remain an explicit option.

Showcase alternative: jellyfish

A softer pink-purple-plum palette that demonstrates the same system can change mood without losing coherence.

The default new-tab shortcuts should point to navi's front doors - neighborli, navi radio, glyph, naviApps, and the navi site - rather than one maintainer's personal history. Personalization begins from a useful public baseline.

Own more of the layer without outrunning the proof

The sequence matters. navi can deepen Chromium integration now while keeping accelera on the horizon rather than turning it into the next-release bottleneck.

Now	Harden the shipped contract. Keep Chromium, the 72 app launchers, local catalog behavior, deterministic dark mode, and current policy supportable on real navi hardware.
Next	Make browser choice explicit. Build the settings TUI, a canonical browser setting, detection, app-mode tests, and clear fallback behavior.
In parallel	Dogfood and push upstream. Measure Helium's update cadence; track Edge Linux issues; verify vendor fixes instead of closing them on response alone.
After proof	Graduate blackice. Replace the uBOL policy entry only after the extension survives normal use and the packaging path is ready.
Later	Begin accelera in sequence. The current roadmap places it after the desktop navi launch, navidroid, and eiri work - a strategic horizon, not the next checkbox.

Non-negotiable principles

- + **Additive, not purist:** a strong default does not require purging alternatives.
- + **Choice preserved:** users choose the browser, while navi owns the compatibility contract.
- + **Zero telemetry:** xvoidsx-owned layers collect nothing by default.
- + **Open models, open web:** agent features should widen the web rather than trap it inside one provider.

Scope boundaries

- Firefox remains on the roadmap as a cross-engine hedge.
- Safari is explicitly out of scope.
- Edge remains optional even if its Linux bugs are fixed.
- Helium remains a candidate base until dogfood evidence is strong enough.
- accelera should not claim integrations before they exist.

Choice does not come from refusing to make decisions. It comes from making the integration legible, keeping alternatives available, and separating what ships from what is merely planned.

The browser is the user's agent on the web

navi treats that role as infrastructure worth understanding and, piece by piece, worth owning.

Today's stack is deliberately practical: Debian Chromium, app-mode windows, local pages, Rofi launchers, managed extensions, and optional alternatives. Tomorrow's stack is more ambitious: a user-selectable Chromium runtime, blackice as an owned privacy layer, and accelera as the browser where navi's visual, smallweb, search, and agent ideas meet.

The continuity matters more than the brand name. navi is not waiting for accelera to begin integrating the browser well, and it is not using today's Chromium dependence as an excuse to avoid owning tomorrow's design.

Key paths in the navi repository

wired/scripts/navi-webapp.sh	Public webapp CLI
wired/naviApps/	Catalog, launchers, icons, local store
wired/chromium/	Policy and dark-mode wrapper
scripts/installers/	Optional browser installers
docs/edge-linux-feedback.md	Public Edge issue tracker

Project links

github.com/xvoidsx/navi
github.com/xvoidsx/blackice

Sources

1. [xvoidsx/navi](#) - repository and eiri working tree.
2. [naviApps local-catalog commit](#) - why catalog data is inlined.
3. [Chrome Enterprise policy reference](#) - force-installed extension behavior.
4. [imputnet/helium](#) and [helium-linux](#) - project, licensing, and Linux packaging.
5. [Microsoft Edge performance features](#) - sleeping tabs and configuration.
6. [navi Edge-on-Linux feedback tracker](#) - issue details and submission log.
7. [xvoidsx/blackice](#) - scope, attribution, license, and supported target.

Status note: "Deployed", "dogfooding", and "planned" labels describe project state as of 23 September 2026. Planned items are not presented as shipped features.